JWT Authentication

a signed token any server can check

Watch the 5-min video

In one line: a JWT is a signed set of claims that any service can verify locally with the issuer's public key, so there's no session lookup; the price is that a token stays valid until it expires, so keep access tokens short and pair them with revocable refresh tokens.

1 Why stateless tokens?

Your API runs on fifty servers across three regions. A session lookup on every request (lesson 40) adds a network hop and a shared dependency in every region. A **JSON Web Token** (RFC 7519) carries the proof itself: if the signature checks out, the claims are true, with no database call. That is why JWTs are everywhere, and why they have sharp edges.

2 Three parts

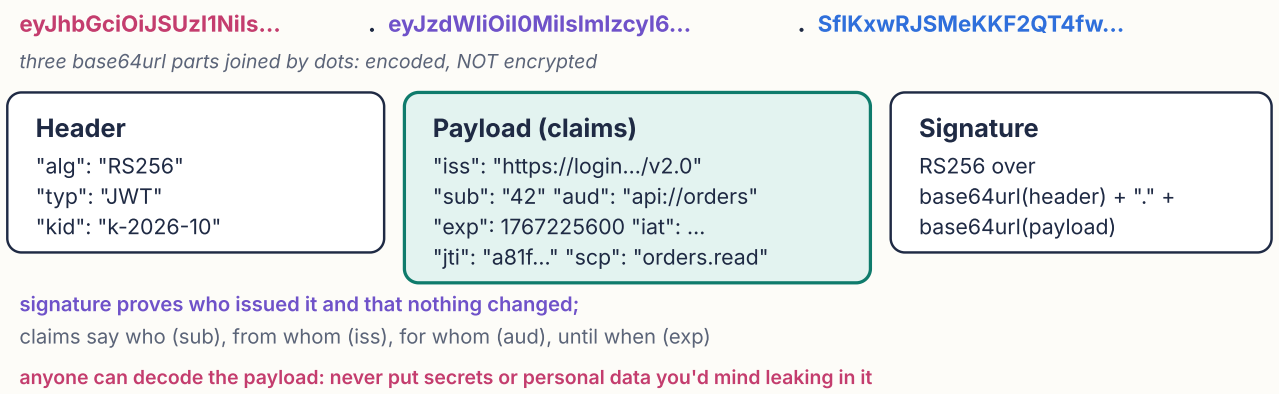


Figure 1. A JWT is `header.payload.signature`. The header names the algorithm and key ID; the payload holds the registered claims.

Registered claims (RFC 7519): `iss`, `sub`, `aud`, `exp`, `nbf` (not-before), `iat`, `jti` (unique token ID). Signed JWTs (JWS) are readable by anyone; JWE encrypts, but is rarely needed.

3 Verify without a lookup

The API (1) checks the signature using the algorithm **it expects**, (2) checks `iss` is the trusted issuer, (3) checks `aud` is this API, (4) checks `exp` / `nbf` with a small clock-skew leeway (about 30–60 s). Only then does it read `sub` and scopes. Verifying one signature takes microseconds, so a gateway can check every request.

4 HS256 vs RS256 / ES256

Algorithm	Key	Who can forge a token
HS256 (HMAC-SHA256)	one shared secret	every service that holds the secret to verify
RS256 (RSA, 2048-bit+)	private / public key pair	only the private-key holder (the IdP)
ES256 (ECDSA P-256)	key pair, smaller keys + signatures	only the private-key holder (the IdP)

Beyond a single service, use asymmetric keys: the identity provider signs, everyone else only verifies.

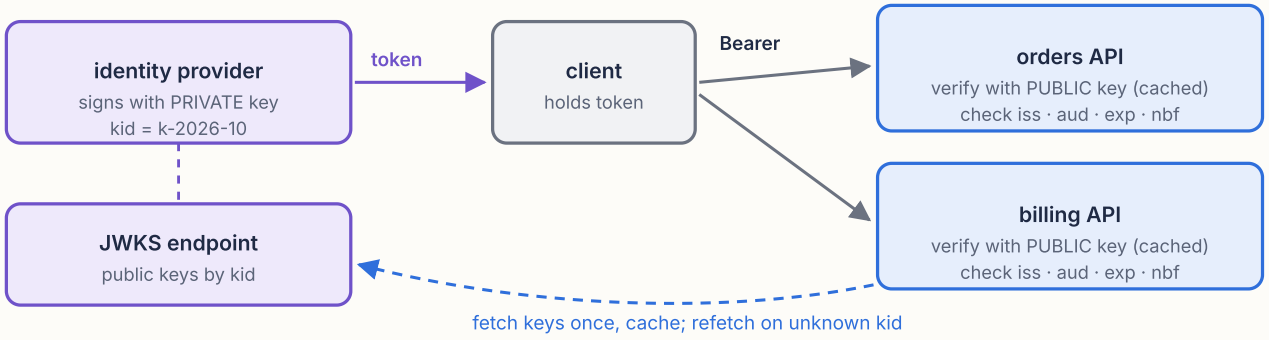


Figure 2. The identity provider signs with its private key and publishes public keys at a JWKS endpoint. APIs verify locally and cache the keys.

5 The revocation problem



Figure 3. A valid JWT stays valid until `exp`, even after logout or a stolen laptop.

So keep **access tokens short-lived** (5–15 minutes) and pair them with a **refresh token** that is stored securely, can be revoked at the identity provider, and is ideally **rotated** on each use (reuse of an old one signals theft). For emergencies, keep a small **deny list of `jti` values** (in Redis, with TTL = remaining lifetime) checked per request. That brings back a lookup, but a tiny one.

6 Key rotation and classic pitfalls

The token header names a `kid`; services cache the JWKS keys and refetch on an unknown one, so rotation needs no redeploy. Publish the new key before signing with it, and keep the old one until its last tokens expire.

COMMON MISTAKES

- Letting the token's header pick the algorithm (`alg: none`, algorithm confusion). Pin it: `algorithms=["RS256"]`.
- Skipping `aud`: a token issued for another API is then accepted by yours.
- Day-long access tokens, or tokens in `localStorage` where any XSS can read them.

WORKED EXAMPLE: FASTAPI VALIDATING ENTRA ID TOKENS

At startup the service reads the tenant's discovery document (`…/<tenant>/v2.0/.well-known/openid-configuration`) and caches the keys from its `jwks_uri` (PyJWT's `PyJWKClient`). Per request it takes the bearer token, picks the key by `kid` and calls `jwt.decode(token, key, algorithms=["RS256"], audience=API_ID, issuer=TENANT_ISSUER)`, which checks signature, `aud`, `iss` and `exp`. No call to Azure per request.

7 Key terms

Claims: statements in the payload: iss, sub, aud, exp, jti

kid: key ID in the header that picks the verification key

Refresh token: long-lived, revocable token used to get new access tokens

JWS: signed token; payload readable by anyone

JWKS: JSON Web Key Set: the issuer's published public keys

Deny list: revoked jti values checked until they expire

8 Interview questions

Q1. JWT vs sessions: what are the trade-offs? BASIC

JWTs are verified locally with no store lookup, so they scale across services and regions, but they're larger and can't be revoked before expiry. Sessions need a shared store lookup per request but revoke instantly. A common hybrid: session cookie for the browser, short-lived JWTs between back-end services.

Q2. How do you revoke a JWT? BASIC

Keep access tokens short-lived (5–15 min), revoke the refresh token, and for emergencies keep a deny list of token IDs (jti) checked on each request.

Q3. Ten microservices must verify user tokens. HS256 or RS256? SCENARIO

RS256 or ES256: the identity provider keeps the private key, services get public keys from JWKS. With HS256 every service holds a secret that can forge tokens.

Q4. How do you rotate signing keys without downtime? SCENARIO

Publish the new public key in the JWKS first, then start signing with it (new kid). Verifiers refetch on unknown kid. Remove the old key after the longest token lifetime has passed.

Q5. Can you put the user's role in the JWT and trust it? TRICKY

Yes, if the identity provider signed it and you verify the signature; but the role is frozen until expiry, so a demotion takes effect only when the token expires. Never trust a role the client sets.

Q6. What is the alg=none or algorithm-confusion attack? TRICKY

If a verifier lets the token header choose the algorithm, an attacker sends an unsigned token (alg none) or an HS256 token keyed with the public RSA key. Pin the expected algorithm and key type.

Next: SD-M-004 · OAuth 2.0 backendroadmap.com